CODING AGENTS AND REVIEW

The review loop



How to check what an agent wrote, and make every review better than the last.

Kenneth Rohde Christiansen

review-loop · open source, Apache-2.0

OPENING

The agent writes the change in minutes. Someone still has to be sure it is right.

This talk is about that someone, and how to stop them being a bottleneck.

OPENING

Where we are going

01

The problem

Writing got cheap. Reading did not.

02

The loop

What a review is made of, and what it costs.

03

Four levels

From one command to a team rule.

04

Learning

Turn every outside comment into a check.

The problem

Writing got cheap. Reading did not.

ACT I · THE PROBLEM

A change you did not type is a change you have to read.

And an agent produces them faster than anyone can read.

ACT I · THE PROBLEM

Two moments to find out a change is wrong

After it goes out

- A reviewer, a bot or a user finds it
- You fix it in another round
- The next change gets the same comment

Before it goes out

- The same checks run first
- It is fixed in the same round
- The class of bug gets a check

ACT I · THE PROBLEM

8 of 9

outside review comments on three pull requests were right.

> **All eight were fixed. None of them changed the next review.** The fixes disappeared into the pull requests.

A real round, in the review-loop README. Copilot comments on three pull requests of one project.

ACT I · THE PROBLEM

Fixing a comment fixes one line. The next change gets the same comment.

Nothing in the process remembers.

ACT I · THE PROBLEM

Most of our time goes on reading, and how we read is almost never discussed.

So it is never improved.

Tudor Girba and Simon Wardley, "Rewilding software engineering", chapter 1, 2025.

ACT I · THE PROBLEM

Teams that review agent-written code at scale review in layers.

Verify before the pull request. An AI review on every pull request, its findings checked by other agents. A file that says what matters. Past incidents as the test set.

Osmani, Lunt and Wang, "Solving code review at Anthropic", October 2026.

ACT I · THE PROBLEM

Not a step at the end. A loop that gets better each round.

Check it yourself. Let others check it. Turn what they find into a check that runs.

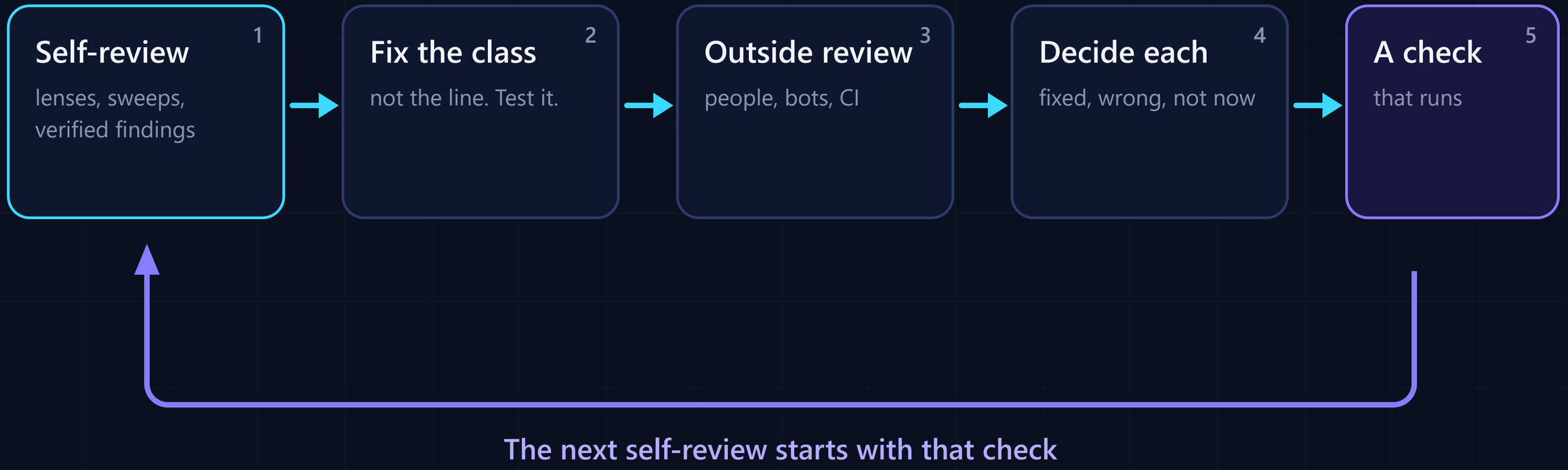
II

The loop

What one review is made of, and what it costs.

ACT II · THE LOOP

Five parts, one loop



ACT II · THE LOOP

A good reviewer reads whole functions, not diff lines.

The callers and siblings needed to answer a question. The report says what was read in full and what was only skimmed.

ACT II · THE LOOP

Nineteen lenses. One question each.

Untrusted input

Missing, null, the wrong type, huge, negative, NaN?

Error paths

Can it report success after the write failed?

State

Every writer of a flag, and every reset of it.

Claims

A comment says "never". Is that true for every caller?

skills/pre-pr-review/SKILL.md, section 3. Others: limits, cost, routing, platform, tests, concurrency, UI, dead code, other doors to the same resource.

ACT II · THE LOOP

A small change, three problems

```
function handle(body, settings) {
  const req = JSON.parse(body);
  const n = parseInt(req.limit);
  const items = req.tags.map((t) => t.trim());
  settings.lastLimit = n;
  return { ok: true, count: items.length, limit: n };
}

function showTitle(el, title) {
  el.innerHTML = title;
}
```

Twelve lines. Read it through the lenses and ask, for each line, what input breaks it.

A small example change made for this talk.

ACT II · THE LOOP

Verify: write the failure, then run it

```
$ node probe.js
{"limit":"1.5","tags":[" a "]} -> limit 1
{"limit":"abc","tags":[]}      -> limit null
{"limit":"5"}                  -> TypeError: reading 'map'
null                           -> TypeError: reading 'limit'
```

Every finding names the input that breaks it. What cannot be made concrete is dropped, or listed as "could not confirm".

Output of the probe on the example change, Node 22.

ACT II · THE LOOP

Outside reviewers are cheap and sometimes wrong. Yours should not be.

A finding without a concrete failure does not go in the report.

37%

of the real problems found by one cheap reader.

> **Our first baseline, code only, the cheapest model for every reader.** A small set of cases. It is the reason for the next slide, not a general result.

review-loop's own first baseline, in skills/pre-pr-review/references/multipass.md.

ACT II · THE LOOP

Three readers, each with one job, then a vote

Reader 1

Input, limits, cost, tests

Reader 2

Error paths, state, concurrency, platform, CI

Reader 3

Agreement, claims, dead code, routing, UI

Merge and vote Found by two readers: it stands. Found by one: confirm it in the code, try to disprove it.

A fourth reader checks the change against an external specification, when REVIEW.md lists one. Readers are not told what the others look for.

ACT II · THE LOOP

Pay for three readers only when it matters

One reader

- Documentation and tests
- A small code change, away from any guard
- Costs one read of the diff

Three readers and a vote

- A larger change
- Always, for a path marked as a guard
- Costs about three reads

review-loop depth, skills/pre-pr-review/scripts/review-depth.sh. Default limit: 60 changed code lines.

ACT II · THE LOOP

It asks before you spend

```
$ review-loop depth HEAD~1 HEAD
depth: single
why: small: 10 code lines (limit 60)

$ review-loop depth HEAD~7 HEAD
depth: multipass
why: 393 code lines (a small change is up to 60)

$ review-loop depth HEAD~20 HEAD
depth: multipass
why: 1731 code lines (a small change is up to 60)
```

The last commit, the last seven and the last twenty of a real game.

Run on Neon Smash (github.com/kenchris/soma-breakout), October 2026.

ACT II · THE LOOP

Three cycles at most. Then ask whether the approach is wrong.

After the first pass, read only the new diff. Fix the class of a bug, not the line.

ACT II · THE LOOP

What failure means here goes in one short file.

REVIEW.md: what counts as a failure, which paths guard something, what to ignore, which classes were raised before. The reviewer reads it first.



Checks that run

Attention is expensive. A search is free.

ACT III · CHECKS THAT RUN

If a search can find the pattern, do not ask a reader to remember it.

A line in a prompt costs attention on every review. A sweep costs a second and no model.

ACT III · CHECKS THAT RUN

Sweeps: the added lines, run through search rules

```
$ review-loop sweeps HEAD~20 HEAD
[json-parse] throws on bad input: is the parsed
  shape checked before use?
  js/assist.js:38:  JSON.parse(localStorage.getItem(...))
[new-listener] is there a matching removal?
  js/secretCode.js:191:  document.addEventListener(...)
diff-sweeps: 15 question(s) for the author (warn-only)
```

No model, no cost. Each hit is a question.

Run on Neon Smash (github.com/kenchris/soma-breakout), last 20 commits, output shortened.

ACT III · CHECKS THAT RUN

A hit is a question, not a finding.

Both of these are fine. The parse sits in a try and clamps the value. The listeners are added once, in a page that lives as long as the game. Answering takes a second.

js/assist.js and js/secretCode.js in Neon Smash.

RECORDED RUN

Depth and sweeps on a real game

RECORDED RUN



Last commit: one reader. Last seven: three. Then the added lines run through the sweeps: two questions, no model.

Windows 11, Git Bash. The output is real; the typing is replayed. Neon Smash, github.com/kenchris/soma-breakout

ACT III · CHECKS THAT RUN

Your own sweep is one line

```
unchecked-parseint ## *.js ## parseInt\([^,)]*\) ## no radix, no NaN check:
  what do "abc" and "1.5" give?
```

name ## which files ## regex ## why. It goes in REVIEW.sweeps at the root of the repository.

The format of REVIEW.sweeps.

ACT III · CHECKS THAT RUN

The gate: "reviewed" becomes a rule

```
$ review-loop gate main HEAD
FAIL: 1 commit(s) are in no review report
  with Open: 0, for example:
  b23e6f4 Add request handler
STOP: run the multipass review and fix
  or explain every finding

$ review-loop gate main HEAD~1
review gate: all 1 commit(s) covered by reports
```

A report in reviews/ with Open: 0 must cover every commit.

Run on the example change made for this talk.

ACT III · CHECKS THAT RUN

The gate checks that a review was written. Not that it was good.

It is a hand-over rule. Quality comes from the lenses, the readers, the verification and the eval.

RECORDED RUN

The gate fails, then passes

RECORDED RUN



No report: it stops. A report with Open: 0 that covers the commit: it passes.

Windows 11, Git Bash. The output is real; the typing is replayed. The report was written by hand.

IV

In practice

Four levels. Stop when it is enough.

ACT IV · IN PRACTICE

Each level adds one thing

01

Ask for a review

One command, no setup.

02

Tell it about your repo

Two minutes, once.

03

Make "reviewed" a rule

For teams.

04

Learn from outside review

Optional, pays off after a few rounds.

ACT IV · LEVEL 1

Ask for a review

```
/pre-pr-review
/pre-pr-review 123
/pre-pr-review main..my-branch
/pre-pr-review multipass
```

The current branch, a pull request, a range, or three readers and a vote. Fix what it finds and run it again.

These are typed to the agent (Claude Code, or VS Code), not to a shell.

ACT IV · LEVEL 1

Install

```
git clone https://github.com/kenchris/review-loop
cd review-loop
./install.sh --link
review-loop doctor

# Windows
.\install.cmd --for all --add-to-path
```

Needs bash and git. On Windows, install.cmd finds the bash of Git for Windows.

README, Install.

ACT IV · LEVEL 2

Tell it about your repository

```
$ review-loop init
created  ./REVIEW.md
created  ./REVIEW.sweeps
created  ./REVIEW.gate
ready    ./reviews/
created  ./.review-loop.env

$ review-loop doctor
8 ok, 0 missing
```

It never overwrites. Edit REVIEW.md a little: the reviewer reads it first.

Run on Neon Smash (github.com/kenchris/soma-breakout).

ACT IV · LEVEL 2

A REVIEW.md for a small game

```
## What a failure is here
- Touch and keyboard doing different things
  for the same input.
- A rule CLAUDE.md describes and the code
  does not do.

## Guards: read in full
- js/secretCode.js  (once per game, never
  in a boss fight)
- sw.js  (a stale cache serves an old game)
```

Short on purpose. A reviewer that has to read a long file reads it badly.

REVIEW.md written for Neon Smash for this talk.

RECORDED RUN

Level 2 in ten seconds

RECORDED RUN



init creates REVIEW.md, REVIEW.sweeps, REVIEW.gate and reviews/. doctor checks the install.

Windows 11, Git Bash. The output is real; the typing is replayed. Neon Smash, github.com/kenchris/soma-breakout

RECORDED RUN

The same checks inside VS Code

RECORDED RUN



Terminal, Run Task, review-loop: sweeps. The hits are added as warnings on the lines, in the Problems panel and in the editor.

VS Code on Windows 11, a clean profile, Neon Smash. Recorded live: the task output and the warnings are real.

ACT IV · LEVEL 3

Make "reviewed" a rule

```
/pre-pr-review multipass my-branch
    -> reviews/feature-limits-b23e6f49.md
       Open: 0

review-loop gate main HEAD
review gate: all 1 commit(s) covered by reports
```

Commit the report. Check before a hand-over or merge.

Run on the example change made for this talk. The report is written by the agent, here by hand.

ACT IV · LEVEL 3

In CI the scripts run. The skills do not.

Sweep hits appear as warnings on the changed lines of a pull request.
The gate can be a required check.

docs/ci.md. The sample workflow is a sketch, not yet run on GitHub.

ACT IV · IN PRACTICE

Plain SKILL.md files. They work where an agent loads skills.

Claude Code, VS Code (Copilot agent mode and the Claude Code extension), other agents that read a skills folder. The scripts need only bash and git.

README, VS Code.

V

Learning

Every outside comment is a check you did not have.

ACT V · LEARNING

From a review thread to the next review

Threads open review comments on GitHub

`threads sync`

Ledger one row per thread, all undecided

`threads worksheet`

Your verdicts fixed, rejected, deferred

`threads verdict`

/retro sorts the misses by cause

Proposals

Add this sweep.
Add this line.
Delete that line.

Nothing changes until you say which to apply.

ACT V · LEARNING

Level 4: four commands

```
review-loop threads sync --repo owner/name
review-loop threads worksheet
review-loop threads verdict <id> fixed --commit <sha>

/retro round 26
```

Download the open threads. See what is undecided. Decide each one. Then ask the agent what would have caught them.

docs/learning-from-reviews.md. Needs node 18 and a token that can read the pull requests.

ACT V · LEARNING

A verdict is your decision about one thread

VERDICT	MEANING
fixed	Right, and fixed. Give the commit.
rejected	Wrong. Say why: it is remembered.
caught	My own review had already reported it.
deferred	Real, but not now.

docs/learning-from-reviews.md. Also: missed, and new for undecided.

ACT V · LEARNING

The cheapest change that would have caught it

1. **A check that runs** — a sweep, a lint rule, a test, a CI job
2. **A pointer** — one line that says where to look
3. **A lens or reader** — a different way of reading
4. **Context** — the spec, the caller list, the config
5. **A REVIEW.md line** — last: only for a judgement call

Take the first that fits.
A hit costs a second.
A line costs attention on every review.

ACT V · LEARNING

What /retro said about a real round

Three

The lens covered it, the reader skipped it.

One

A one-line search would find it: a pointer cast with no family check.

One

A deleted test file nobody had noticed.

Result

A ranked list of changes, instead of nine fixes that teach nothing.

The Copilot round from slide 7: nine comments, eight right. The README does not break down the other three.

ACT V · LEARNING

Measure the reviewer too. Recall and noise. Both.

Recall: of the real problems, how many it reported. Noise: of what it reported, how much was wrong. Raising recall is easy. Noise is what it costs the author.

skills/review-eval/SKILL.md.

ACT V · LEARNING

A rejected comment keeps your reason. It is not fixed twice.

And a change to the lenses is kept only if the eval says it helped.

ACT V · LEARNING

It pays off after a few rounds. With one or two comments there is nothing to learn from yet.

Start with review-loop threads sync.

VI

Limits

What it does not do.

ACT VI · LIMITS

A small, generic version of some of these pieces

It does

- Lenses, sweeps and verification
- Several readers and a vote
- A report, a gate and a score
- Retro and eval

It does not

- Merge queues and staged rollout
- Feature flags
- Risk-tiered approval (planned)
- Explainers for modules (planned)

README, Background, and PLAN.md.

ACT VI · LIMITS

Three readers cost about three times one. Say so before launching.

The tool asks first for a large diff, suggests a model per reader, and the eval records the tokens each run used.

ACT VI · CLOSE

On Monday

```
review-loop depth main HEAD
review-loop sweeps main HEAD
/pre-pr-review
```

Free, free, and one reader. Fix what it finds. Stop here if that is enough.

Levels 1 and 2 of the README.

ACT VI · CLOSE

Start at level 1. Stop when it is enough.

github.com/kenchris/review-loop

VII

Hands-on

On your own branch. About 20 minutes.

HANDS-ON

Six steps

01

Install

review-loop doctor shows 8 ok.

02

Ask

/pre-pr-review on a branch with changes.

03

Compare

review-loop depth, then sweeps.

04

Setup

review-loop init, edit REVIEW.md.

05

Rule

Save a report, run the gate.

06

Learn

Optional: threads sync, then /retro.

HANDS-ON · STEPS 1 TO 3

Install, ask, compare

```
./install.sh --link
review-loop doctor

/pre-pr-review

review-loop depth main HEAD
review-loop sweeps main HEAD
```

Did the review find something the sweeps did not? Did the sweeps ask something the review answered?

HANDS-ON · STEPS 4 AND 5

Setup, then the rule

```
review-loop init
review-loop doctor

/pre-pr-review multipass my-branch
review-loop gate main HEAD
```

Edit REVIEW.md first: three lines on what failure means in your project.

HANDS-ON · STEP 6

For those with open pull requests and review comments.

review-loop threads sync --repo owner/name, then decide each thread.
Then /retro, and read the list before you apply anything.